POLICYPILOT

SOFTWARE DESIGN DOCUMENT

A Permission-Aware, Evaluated Enterprise Knowledge Assistant

Prepared by
Abdul Rauf

Version 1.0 **Date** 17 October 2026

Basis PolicyPilot Revised Project Proposal (16 Oct 2026) and SRS v0.1 Draft (16 Oct 2026)

Design principle

Authorization is enforced before any retrieved passage enters the language-model context. Verification improves answer quality but is never treated as a security boundary.

DESIGN STATUS: Proposed implementation baseline. Open decisions are explicitly recorded; this document does not claim that the system has already been implemented or security-certified.

Document control

Field	Value
Document	Software Design Document (SDD)
Product	PolicyPilot — Permission-Aware Enterprise Knowledge Assistant
Version / status	1.0 / Proposed design baseline for review
Date	17 October 2026
Author	Abdul Rauf
Source requirements	PolicyPilot Revised Project Proposal; PolicyPilot SRS v0.1 Draft
Audience	Developer, reviewer, evaluator, academic supervisor, future maintainer
Normative language	“Shall” indicates a required design behavior; “should” indicates a recommended practice.

Revision history

Version	Date	Change
1.0	17 Oct 2026	Initial detailed design derived from the supplied proposal and SRS; diagrams, interfaces, data model, security controls, test strategy, and open decisions added.

How to use this document

This SDD describes the proposed architecture and the design decisions needed to implement and evaluate PolicyPilot. The SRS remains the source of functional and non-functional requirements. Where the SRS leaves a choice open, this document marks it as a proposed decision or TBD rather than silently presenting it as an established fact.

Contents

- 1. Purpose, scope, and design goals
- 2. System context and actors
- 3. Architecture overview
- 4. Core workflows
- 5. Component design
- 6. Security and authorization design
- 7. Data design
- 8. API design
- 9. Error handling and status model
- 10. Quality attributes and operational design
- 11. Evaluation and test design
- 12. Deployment and CI/CD
- 13. Design decisions and trade-offs
- 14. Requirements traceability
- 15. Risks, limitations, and open decisions
- Appendix A. Example query contract
- Appendix B. Review checklist

1. Purpose, scope, and design goals

1.1 Purpose

Define an implementable design for PolicyPilot, a retrieval-augmented generation (RAG) assistant that answers questions from approved handbook/policy content, cites supporting sources, applies server-side authorization, verifies claims, abstains when evidence is insufficient, and exposes reproducible evaluation and operational measurements.

1.2 In-scope capabilities

• Approved-document ingestion, heading-aware chunking, metadata validation, embedding generation, indexing, and stale-content removal.

• Demo-user authentication; content permissions and system roles kept as separate concepts.

• Permission-filtered keyword/vector hybrid retrieval, cross-encoder reranking, evidence-linked generation, claim verification, abstention, and basic escalation status.

• Web UI, REST API, evaluation/security suites, tracing, latency and cost reporting, automated tests, Docker Compose, and CI.

1.3 Explicitly out of scope

Model fine-tuning or custom embedding-model training; real enterprise identity-provider integration; production certification; multi-tenant hardening; and a full human case-management workflow. Urdu/bilingual queries, agentic retrieval, and table-aware/scanned-document ingestion are possible extensions only after the core system is stable.

1.4 Design priorities

Priority	Design implication
P0 — Authorization correctness	Deny by default; filter inside retrieval; re-authorize before context assembly; protect citations, caches, errors, and logs.
P1 — Reproducibility	Freeze configuration for baseline/final runs; pin package and model identifiers; record prompts and parameters.
P2 — Answer reliability	Citations, claim-level evidence checks, calibrated abstention, and measured false rejections.
P3 — Operability	Trace stages, record timings/token cost, test in CI, and provide deterministic setup instructions.
P4 — Presentation polish	Keep UI simple until security and evaluation criteria are met.

1.5 Key architectural invariants

• Client-supplied roles or permissions are never authoritative.

• No unauthorized passage may be sent to the generation or verification model.

• Retrieved document text is untrusted data and cannot override system policy or authorize tool actions.

• Every factual answer claim should map to authorized evidence; unsupported claims are qualified, removed, or cause abstention.

• System roles (Admin/Evaluator) do not automatically grant content access.

• Evaluation results are evidence for tested scenarios, not proof of universal security.

2. System context and actors

Actor	Purpose	Authorization rule
Employee / Manager / HR / Finance	Ask policy questions and open citations.	Content access comes from explicitly assigned content permissions; users can hold multiple permissions.
Admin	Manage demo users/permissions and start or inspect ingestion runs.	System feature access only; restricted content access requires separate content permission.
Evaluator	Run evaluation/security suites and view reports.	Evaluation feature access only; content access follows assigned permissions.
Maintainer	Deploy, configure, inspect health and sanitized telemetry.	Operational access is controlled separately; no implicit right to restricted document content.

2.1 Context diagram

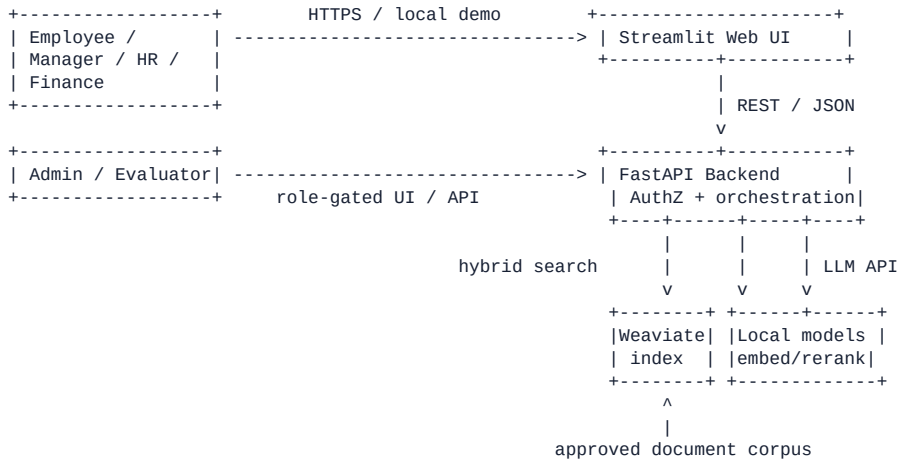


Figure 1. System context. The backend, not the browser, is the trust boundary for authentication, permissions, and query orchestration.

2.2 Permission model

Content permissions are Public, Employee, Manager, HR, and Finance. Public content is available to all users; Employee is the baseline for authenticated users. Additional categories are independent and additive: Manager does not imply HR or Finance. A user with Employee + Manager + HR may read Public, Employee, Manager, and HR content, but not Finance content. Admin and Evaluator are system roles, not content permissions.

Content permission	Employee	Manager	HR staff	Finance staff
Public	Allow	Allow	Allow	Allow
Employee	Allow	Allow	Allow	Allow
Manager	Deny	Allow	Deny	Deny
HR	Deny	Deny	Allow	Deny
Finance	Deny	Deny	Deny	Allow

The matrix is an illustrative project policy from the supplied documents. Multi-permission users receive the union of explicitly assigned permissions. Resolve any ambiguous identity/permission state by denying access.

3. Architecture overview

3.1 Logical architecture



Figure 2. Logical flow. The authorization re-check occurs before context assembly and model invocation. Verification is a quality-control step, not an authorization mechanism.

3.2 Proposed technology baseline

Concern	Baseline choice	Status / rationale
Language/API	Python 3 + FastAPI	Specified by SRS.
Web UI	Streamlit	Selected for MVP simplicity and Python-only implementation.
Search/index	Weaviate hybrid BM25 + vector	Selected in SRS for native hybrid search and metadata filtering.
Embeddings	One open-source embedding model	Exact model ID is TBD; benchmark and freeze before final evaluation.
Reranking	One cross-encoder	Exact model ID is TBD; compare on fixed evaluation set.
Generation / verification	Hosted LLM API	Provider, model ID, prompt versions and parameters TBD; budget cap required.
Deployment / CI	Docker Compose + GitHub Actions	Pin dependencies; automated test gates.
Observability	Structured logs + tracing tool	Tool and retention period TBD; avoid sensitive text in telemetry.

3.3 Trust boundaries

• TB-1 Browser → API: treat all client input as untrusted; validate question length, session, and request shape.

• TB-2 API → index: enforce metadata filters server-side and independently re-check every candidate.

• TB-3 Retrieved text → LLM: content is untrusted evidence, never executable instructions; no tools are granted by retrieved text.

• TB-4 API → hosted LLM: send only authorized, minimized context; use TLS and secrets from environment/secret management.

• TB-5 Logs/cache → later requests: sanitize logs and isolate cache entries by effective permission set and relevant configuration.

4. Core workflows

4.1 Query sequence

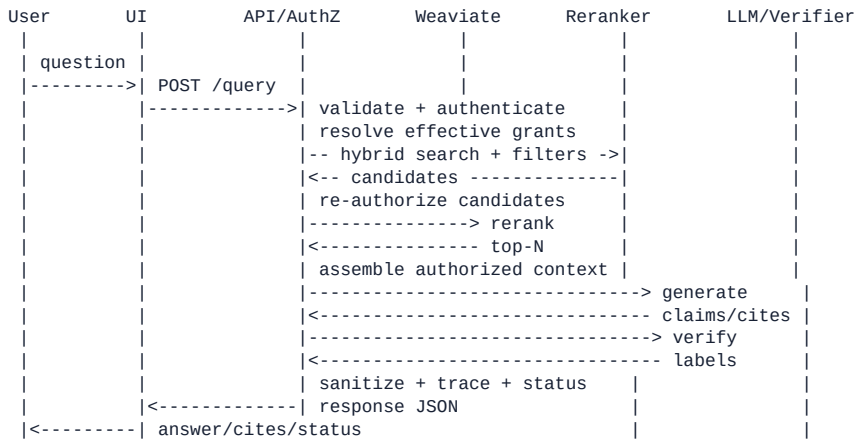


Figure 3. Sequence diagram for the main query. A failure to authenticate or resolve permissions stops the flow before retrieval.

4.2 Ingestion workflow

• 1. Admin starts an ingestion run for an approved source set.

• 2. Fetch source and record URL, retrieval date, licence note, and version.

• 3. Parse text while preserving headings/sections; record parsing failures.

• 4. Split into heading-aware chunks with documented size/overlap; assign stable document and chunk IDs.

• 5. Validate required metadata and allowed permission labels. Reject invalid chunks rather than indexing them.

• 6. Generate embeddings and upsert chunks into Weaviate with keyword-searchable text and permission metadata.

• 7. For updated/deleted documents, replace or remove all prior chunks to prevent stale indexed content.

• 8. Validate citation links and metadata; mark run successful only after required checks pass.

4.3 Verification and abstention workflow

• Generate answer claims with citations to document/section/chunk identifiers.

• Split the response into factual claims; for each claim, check the cited evidence.

• Assign Supported, Contradicted, or Insufficiently Supported labels.

• Keep supported claims; qualify or remove unsupported claims; if remaining evidence is insufficient, return ABSTAINED or ESCALATED.

• Record verifier model/configuration and its outcome, but never use the verifier to decide authorization.

5. Component design

Component	Responsibilities	Inputs → outputs
Streamlit UI	Login; question form; answer/status; citation list; Admin and Evaluator pages gated for usability.	User actions → REST requests and rendered response.
API router	Request validation, authentication dependencies, response schemas, status codes.	HTTP request → typed response.
Authentication service	Verify credentials; issue/validate session token; load current user state.	Credentials/token → principal or deny.
Authorization service	Resolve grants and system roles from server-side store; deny by default; evaluate chunk access.	Principal + chunk metadata → allow/deny.
Ingestion service	Fetch/parse/chunk documents; validate metadata; embed; upsert/delete index entries.	Approved source → ingestion run + indexed chunks.
Retrieval service	Apply hybrid search and permission filters; deduplicate/fuse candidate sets; re-authorize candidates.	Question + grant set → authorized candidates.
Reranker	Rescore permitted candidates and select documented top-N.	Question + candidates → ranked passages.
Answer service	Build bounded context and prompt; generate cited response using only authorized evidence.	Question + evidence → draft claims/citations.
Verifier / abstention	Check claims against cited passages; apply calibrated threshold and response policy.	Draft + evidence → supported/partial/abstained result.
Evaluation service	Run labelled benchmark and security scenarios; compute metrics; persist configuration snapshots.	Dataset + config → results/report.
Observability	Stage timings, request IDs, token usage/cost estimates, sanitized structured logs.	Events → traces/metrics without restricted text.

5.1 Component contracts

Each service should have a narrow interface and be unit-testable without the hosted LLM. Authorization should be a reusable backend service rather than duplicated ad hoc checks. Retrieval must return candidate identifiers and metadata sufficient for a second authorization check. Answer generation must accept an already-authorized evidence bundle, not a raw database query handle.

5.2 Suggested package structure

```
policypilot/
  app/
    main.py                 # FastAPI application and lifespan
    api/
      auth.py               # /auth/login, /me
      query.py              # /query, /sources/{chunk_id}
      admin.py              # ingestion and permission administration
      evaluation.py         # evaluation runs and reports
      health.py             # /health
      schemas.py            # Pydantic request/response contracts
    core/
      config.py             # environment-based settings
      security.py           # password/token primitives
      logging.py            # redaction and structured logging
    services/
      authentication.py
      authorization.py
      ingestion.py
      retrieval.py
      reranking.py
      answer_generation.py
      verification.py
      abstention.py
      evaluation.py
    repositories/
      users.py
      documents.py
      runs.py
      search_index.py       # Weaviate adapter
    models/
      domain.py             # User, Document, Chunk, QueryResult
    ui/
      streamlit_app.py
      pages/
        admin.py
        evaluator.py
    tests/
      unit/
      integration/
      authorization/
      security/
      evaluation/
  evaluation/
    datasets/               # labelled data and split manifest
    configs/                # frozen experiment configurations
    reports/
  docs/
    architecture.md
    decisions/
  Dockerfile
  compose.yaml
  pyproject.toml
  .env.example
  README.md
```

The exact repository layout is a design recommendation, not a requirement already fixed by the SRS.

6. Security and authorization design

6.1 Authentication and authorization sequence

1. Validate request shape and session token.
2. Resolve the current principal from server-side user records.
3. Load current content_permissions and system_roles (do not trust browser claims).
4. If principal/grants cannot be resolved -> deny safely.
5. Build effective content grants = Public + Employee baseline + explicit assigned categories.
6. Run retrieval with a metadata filter for those grants.
7. Re-check each candidate against current grants before it can enter context.
8. Re-check again before returning a source passage or citation target.
9. Prevent stale authorization through permission-versioning/cache invalidation.

6.2 Authorization decision rule

A chunk is eligible only when its allowed_permissions intersects the user's effective content permissions. The Public and Employee baseline behavior is explicit policy; the implementation should normalize these grants consistently in one authorization service. Empty or malformed permissions, missing identity, lookup failure, and unknown permission labels must result in denial.

```
def can_read(user_grants, chunk_permissions):
    if not user_grants or not chunk_permissions:
        return False
    if not permissions_are_valid(user_grants, chunk_permissions):
        return False
    return bool(set(user_grants).intersection(set(chunk_permissions)))
```

Illustrative pseudocode only. Implementations must normalize the baseline grants consistently and test all combinations, including multi-permission users.

6.3 Security controls

Threat / failure mode	Required control	Verification evidence
Client forges role/permission	Resolve grants from server-side identity; ignore client-provided role fields.	API tests mutate role fields and confirm unchanged access.
Restricted content retrieved	Metadata filter in retrieval plus independent candidate re-check before context assembly.	Negative tests inspect retrieval/context boundary.
Indirect inference from restricted-only evidence	Return a response indistinguishable from no evidence; avoid leaking titles/counts/snippets.	Paired tests for restricted-only versus no-evidence cases.
Citation/source endpoint leakage	Authorize each chunk fetch; return not-found for unauthorized sources.	SEC-03 and direct source endpoint tests.
Cross-user cache leakage	Cache key includes effective permission set/version and relevant model/prompt/index configuration; invalidate on permission changes.	SEC-04 and SEC-05.
Prompt injection in document	Delimit retrieved passages as untrusted evidence; ignore embedded instructions; no tool execution from retrieved text.	SEC-06 attack suite.
Fabricated evidence/citations	Only cite retrieved chunk IDs; verify claims against cited evidence; reject unknown IDs.	SEC-07 and citation-integrity tests.
Permission lookup failure	Fail closed; do not invoke LLM or return evidence.	SEC-08.
Admin/Evaluator privilege confusion	Separate system feature authorization from content permission checks.	SEC-09.
Stale index content	Versioned ingestion and delete/replace all chunks on update/removal.	Ingestion update/delete integration tests.
Secrets and sensitive logs	Environment-based secrets, no committed keys, redact inputs and restrict logs.	Secret scanning and log-content tests.

6.4 Prompt construction principles

• Keep system policy separate from retrieved content; label retrieved passages as untrusted quotations/evidence.

• Require citation identifiers to be selected from the supplied evidence set; never permit the model to invent a source ID.

• Ask for concise answers and explicit uncertainty when evidence is incomplete.

• Do not expose credentials, environment variables, internal prompts, hidden metadata, or unauthorized passages.

• Do not grant external tools to the model for the MVP; any future tool integration requires separate authorization and threat modelling.

6.5 Session, password, and API hardening

Use salted password hashing through a maintained password-hashing library; never store plaintext passwords. Sign session tokens with a strong secret loaded from environment/secret storage, set expiry, validate issuer/audience as applicable, and avoid putting sensitive permissions in long-lived client-controlled claims. Apply request size limits, input validation, rate limits suitable for a demo, HTTPS for hosted deployments, and safe generic authentication errors. Exact token mechanism and expiry remain TBD.

6.6 Security scope statement

A clean test run demonstrates only that the tested scenarios did not show a leak. It does not prove universal security, production readiness, or resistance to every prompt-injection strategy.

7. Data design

7.1 Logical entities

Entity	Important fields	Rules / notes
Document	doc_id, title, source_url, retrieved_on, licence_note, version, allowed_permissions, updated_at	Source lineage and permission labels are required; keep source version.
Chunk	chunk_id, doc_id, section, text, embedding, version, updated_at, allowed_permissions	Stable IDs; permission metadata inherited from document unless explicitly overridden; validate before index.
User	user_id, username, password_hash, content_permissions[], system_roles[], permission_version	System roles and content permissions are separate.
Evaluation item	item_id, question, category, expected_answer or expect_abstain, relevant_chunk_ids[], permitted_roles[], split	Keep development and frozen final sets distinct.
Run record	run_id, type, configuration_snapshot, metrics, started_at, completed_at, status	Store ingestion/evaluation status and failures.
Request log	request_id, user_id, status, timings, token_usage, estimated_cost, timestamp	No restricted passage text or secrets; retention TBD.

7.2 Index metadata and filter

Each indexed chunk should carry stable document/chunk identifiers, source URL, section, source version, update timestamp, and normalized allowed_permissions. Use a metadata filter in the same hybrid retrieval request. Re-check candidate permissions after retrieval because filters can be misconfigured or metadata can be stale.

7.3 Data lifecycle and integrity

• Use deterministic IDs or a stable source/version mapping so re-ingestion updates rather than duplicates chunks.

• Record source hash/version to identify changed content and support repeatable ingestion.

• Make ingestion runs idempotent where feasible; failed runs should expose sanitized failure details and not be reported as successful.

• On update or deletion, remove old chunks and embeddings; test that no stale chunk remains retrievable.

• Validate citation URL/section mappings and permission metadata before finalizing a run.

• Retain dataset provenance and licensing notes; do not redistribute processed data unless terms allow it.

7.4 Cache design

Caching is optional for the MVP. If enabled, a response-cache key must include the normalized effective permission set and permission version, query normalization, index version, prompt/model configuration, and relevant generation settings. Permission changes must invalidate or bypass old entries. Do not cache raw restricted passages in a shared cache without equivalent access controls.

7.5 Retention and privacy

The SRS does not finalize a retention period. Before deployment, decide retention for request traces, evaluation artifacts, and user records. Prefer minimal logs: request IDs, stage timings, status, token counts, cost estimates, and error codes. Avoid logging full questions or retrieved passages by default; if debugging requires samples, use an explicitly controlled, redacted path.

8. API design

8.1 API conventions

• REST/JSON; versioning strategy should be fixed before public clients depend on the API (proposed prefix /api/v1).

• Validate inputs and outputs with typed schemas; reject unexpected fields where practical.

• Use consistent error objects with a request_id and safe message; never include restricted content in error details.

• Enforce authorization in API dependencies and service methods, not only by hiding UI controls.

• Document endpoints with FastAPI OpenAPI output; keep schemas backward-compatible within a version.

8.2 Proposed endpoint catalogue

Method / endpoint	Access	Purpose	Key behavior
POST /auth/login	Public	Authenticate demo user and create session.	Generic invalid-credential response; rate-limit attempts.
GET /me	Authenticated	Return current identity, effective content permissions, system roles.	Read grants from server-side records.
POST /query	Authenticated	Ask question and return answer/citations/status.	Authorize retrieval and context; trace request.
GET /sources/{chunk_id}	Authenticated	Open cited passage.	Return not-found when absent or unauthorized.
POST /admin/ingest	Admin	Start ingestion run.	Role gate plus input validation.
GET /admin/ingest/{run_id}	Admin	View run status and sanitized failures.	No source secrets or restricted text in error.
PUT /admin/users/{id}/permissions	Admin	Update content permissions.	Validate known permissions; bump permission version/invalidate cache.
POST /eval/runs	Evaluator	Start evaluation/security suite.	Record configuration snapshot and dataset split.
GET /eval/runs/{run_id}	Evaluator	Fetch metrics/report.	Only authorized evaluators; avoid leaking source text.
GET /health	Public	Health check.	Expose minimal status, no secrets/config dump.

8.3 Query request / response

```
POST /query
{
  "question": "What is the annual leave policy?",
  "request_options": {
    "include_citations": true
  }
}

200 OK
{
  "request_id": "req_...",
  "status": "ANSWERED",
  "answer": "The policy states ...",
  "citations": [
    {
      "chunk_id": "chunk_...",
      "doc_id": "doc_...",
      "section": "Leave / Annual leave",
      "source_url": "https://example.invalid/handbook",
      "quote": "Authorized supporting excerpt..."
    }
  ],
  "verification": {
    "supported_claims": 2,
    "removed_or_qualified_claims": 0
  }
}
```

Illustrative contract only; the actual corpus and citation URL are not assumed. Citation quote text must be authorized for the current user. In a stricter privacy configuration, return only section/source metadata and load the passage through the separately authorized source endpoint.

8.4 Status and HTTP mapping

Application status	Typical HTTP status	Meaning
ANSWERED	200	Answer passed configured evidence checks.
PARTIAL	200	Some claims were removed or qualified; remaining response is usable.
ABSTAINED	200	Evidence is insufficient; no unsupported answer is asserted.
ESCALATED	200	Abstention plus suggested next step; no case-management workflow implied.
ERROR	4xx/5xx as appropriate	Request or service failed; safe error body includes request_id.

9. Error handling and status model

9.1 Failure behavior

Failure	Expected behavior
Invalid request / oversized question	Return validation error; do not call retrieval or LLM.
Invalid or expired session	Return authentication error; no retrieval.
Permission service unavailable / malformed grants	Fail closed; do not invoke LLM.
Search database unavailable	Return safe service error; trace failure stage; do not fabricate an answer.
No authorized evidence	ABSTAINED or ESCALATED; do not disclose whether restricted content exists.
Reranker/LLM/verifier timeout	Bound retries; return safe ERROR or configured fallback that does not bypass authorization/verification.
Malformed model output / unknown citation IDs	Reject or repair under a bounded policy; never return unvalidated source identifiers.
Partial ingestion failure	Report failed items; do not mark successful unless required validations pass.

9.2 Retry and timeout policy

Use explicit per-stage timeouts and a total request deadline. Retry only transient failures, with bounded attempts and backoff. Do not retry non-idempotent administrative actions without an idempotency key. If verification fails, do not silently downgrade to unverified generation; return a documented safe status or abstain. Numeric timeout values should be set from baseline measurements and recorded in configuration.

10. Quality attributes and operational design

Attribute	Design approach	Measurement / acceptance
Security	Deny-by-default; filtered retrieval; candidate re-check; source authorization; permission-aware cache.	No observed leaks in defined suite; report attempts and coverage.
Answer quality	Hybrid retrieval, reranking, evidence-linked claims, verification, abstention.	Recall@5, MRR, Precision@k, correctness, unsupported claim rate, citation correctness.
Performance	Bound context size; fixed top-N; stage-level timings; bounded retries.	p50/p95 with and without verification; proposed p95 ≤ 10 s is TBD until baseline.
Cost	Token usage tracking, embedding cache, budget caps, limited experiments.	Estimated cost per 1,000 queries and per request.
Reliability	Health endpoint, safe failures, idempotent ingestion, CI checks.	Automated unit/integration/security/regression results.
Maintainability	Service boundaries, typed schemas, configuration, decision log.	Reviewable modules; pinned dependencies; test coverage report.
Reproducibility	Freeze dataset split, model/prompt IDs, parameters and package versions.	Independent engineer can reproduce baseline and final run.
Privacy	Minimal sanitized logs, no restricted text in logs/errors, documented retention.	Automated log scanning and retention decision.

10.1 Observability events

Span / event	Suggested fields	Do not include
request	request_id, timestamp, user_id pseudonym, status	Password, token, full restricted content
authorization	decision, permission version, candidate count	Restricted titles/snippets in broadly accessible logs
retrieval	mode, latency, candidate count, index/config version	Unauthorized candidate text
reranking	model ID, latency, input/output counts	Full passages unless protected debug capture is approved
generation/verification	model ID, prompt version, latency, token counts, outcome	Secrets or unrelated private data
cost summary	input/output tokens, estimated provider cost, config ID	API keys or billing credentials

10.2 Performance test protocol

Record hardware/runtime, concurrency, corpus/index version, question set, model IDs, warm/cold state, and whether verification is enabled. Report sample count and p50/p95 latency, plus estimated cost per 1,000 queries. Fix numeric acceptance targets before the frozen final evaluation; do not choose thresholds after seeing the final results.

11. Evaluation and test design

11.1 Evaluation dataset

Start with approximately 60 labelled questions spanning answerable/unanswerable cases, exact-term questions, multiple-relevant-passage questions, and role-restricted cases. Each item records expected answer or abstention, relevant source passages, permitted roles, and category. This is a small development benchmark; conclusions are preliminary unless supported by additional independent tests.

11.2 Test layers

Layer	Examples	Runs when
Unit	Permission normalization; policy intersection; chunk IDs; schema validation; abstention threshold.	Every commit.
Integration	Weaviate filters; ingestion update/delete; API auth; authorized source endpoint; cache invalidation.	Every commit / PR.
Security regression	SEC-01 to SEC-10; prompt injection; role forgery; restricted-only inference; logs and cache.	Every PR subset; full suite before final report.
RAG quality	Recall@5, MRR, Precision@k; citation support; claim correctness; abstention trade-off.	Baseline and controlled experiments.
End-to-end	Login → query → cited answer/abstention; Admin ingestion; Evaluator run and report.	Release candidate.
Operational	Health, timeout, model outage, cost budget, p50/p95.	Baseline and final evaluation.

11.3 Security scenario checklist

ID	Scenario	Expected result
SEC-01	User requests restricted document by exact title.	No restricted passage/title disclosure.
SEC-02	Indirect question tries to reveal restricted information.	No information derived solely from unauthorized evidence.
SEC-03	Answer has citations.	Every cited source is authorized.
SEC-04	Cached result requested under a different permission set.	Cache miss/separate entry; no cross-permission response.
SEC-05	Permission is removed.	Next request observes removal; stale access denied.
SEC-06	Retrieved content says to override policy or reveal secrets.	Text treated as untrusted; system policy retained.
SEC-07	Document tries to induce fabricated evidence/citations.	Unknown or unsupported citations rejected/qualified.
SEC-08	Authentication or permission resolution fails.	Fail closed before model context is created.
SEC-09	Admin/Evaluator lacks content permission.	System role alone does not grant document access.
SEC-10	Manager + HR user requests content.	Union of grants; Finance remains inaccessible.

11.4 Required metrics

Metric	Reporting approach
Recall@5 / MRR / Precision@k	Compare retrieved candidates to labelled relevant passages.
Answer correctness	Compare to expected answer; manually review a sample.
Unsupported claim rate	Manual claim labels; compare before/after verifier.
Citation correctness	Check whether cited passages support claims and are authorized.
Abstention precision and coverage	Report correct abstentions and answer coverage; include false refusals.
Observed leakage	Number of successful leaks / total tests, with scenario coverage.
Injection success rate	Successful attacks / total attempts, by documented category.
Latency and cost	p50/p95 and estimated cost per 1,000 queries.

11.5 Experimental controls

• Freeze baseline chunking, retrieval configuration, models, prompts, parameters, corpus and evaluation split.

• Change one major variable at a time where practical; keep other settings fixed.

• Keep a held-out final set separate from prompt/threshold tuning; if too small, disclose this limitation.

• Manually review a sample of labels; treat LLM judges as secondary aids, not ground truth.

• Report counts with percentages; include confidence intervals where useful; publish negative results and failure cases.

• Version every evaluation run with configuration and dataset hashes.

12. Deployment and CI/CD

12.1 Local deployment topology

```
Developer machine / demo host
  Docker Compose
   |-- api        FastAPI service
   |-- ui         Streamlit service
   |-- weaviate   Search/index service
   |-- optional telemetry/tracing service (choice TBD)
   +-- model files mounted/cached locally where appropriate

External dependencies:
  - hosted LLM API over TLS
  - approved public handbook source(s) during ingestion
```

Figure 4. Proposed deployment. The exact container topology, model resource needs, and tracing service are implementation decisions to validate during the baseline.

12.2 CI pipeline

```
Pull request
  -> formatting / lint / type checks
  -> unit tests
  -> integration tests (service containers where feasible)
  -> authorization and security regression subset
  -> evaluation smoke subset
  -> dependency / secret scan
  -> build container image
  -> publish test results and configuration metadata
```

12.3 Release gates

• All required unit and API tests pass.

• Authorization tests include negative cases and source/citation/cache checks.

• No secrets are committed; hosted demo uses HTTPS and synthetic users/public content only.

• Dependencies and model IDs are pinned/documented; .env.example contains placeholders only.

• Baseline and final evaluation can be reproduced using README instructions.

• Known limitations and observed security-test coverage are published with the demo.

12.4 Configuration and secrets

Use environment-based settings and a checked-in .env.example containing non-secret placeholders. Never commit real API keys, token-signing secrets, passwords, or private corpora. Separate local/test/demo configurations. Set an LLM usage budget and bounded evaluation run counts. Pin package versions and record model identifiers and prompt versions.

13. Design decisions and trade-offs

ID	Decision	Rationale	Trade-off / revisit when
ADR-001	FastAPI + Streamlit for MVP.	Fast implementation; single-language stack.	Less customized UI; revisit only after core controls work.
ADR-002	Weaviate as initial search database.	SRS selects native hybrid BM25/vector search with metadata filtering.	Validate filter behavior and test it; reconsider only with measured evidence.
ADR-003	Server-side deny-by-default authorization.	Prevents client roles and model output from becoming authority.	More checks and tests; mandatory for this project.
ADR-004	Re-check candidates before LLM context.	Defense in depth against filter/config/metadata failure.	Small added latency; measure it.
ADR-005	Verification separate from authorization.	Verifier may be wrong or variable; it cannot be security control.	Extra LLM cost/latency; measure benefit and false rejection.
ADR-006	Permission-aware caching is optional.	Avoids cross-permission leakage; caching is not required for MVP.	May reduce cache hit rate; omit cache if complexity outweighs value.
ADR-007	Freeze baseline before controlled comparisons.	Makes experiments interpretable and reproducible.	Slower iteration; use config snapshots and a separate dev set.
ADR-008	No agent tools in MVP.	Reduces attack surface from untrusted retrieved content.	Less agentic behavior; future tools require independent authorization design.

13.1 Decision log format

For every major change record: decision ID, date, context, alternatives considered, decision, security implications, measured result, cost/latency impact, and conditions for revisiting. A design change should not be called an improvement unless it is measured against the fixed protocol or its limitation is clearly stated.

14. Requirements traceability

Objective / requirement group	Design sections	Verification evidence
Document ingestion (FR-ING)	4.2, 5, 7	Ingestion integration tests; update/delete stale-chunk test.
Authentication/authorization (FR-AUTH)	2.2, 6, 8	SEC-01–SEC-10; negative permission tests.
Hybrid retrieval/reranking (FR-RET)	3, 4.1, 5	Recall@5/MRR/Precision@k and latency comparisons.
Generation/verification/abstention (FR-GEN/VER)	4.3, 5, 9	Citation correctness, unsupported claim rate, false rejection/abstention results.
UI/admin/evaluator (FR-UI)	2, 8, 12	End-to-end role-gated flows.
Evaluation (FR-EVAL)	11	Reproducible runs, metrics and security reports.
Observability (FR-OBS)	10	Trace completeness, timing, sanitized log checks.
Performance (NFR-PERF)	10.2	Documented p50/p95 test and cost report.
Security (NFR-SEC)	6, 7, 12	Secret scan, password hash review, HTTPS for hosted demo.
Reproducibility/maintainability (NFR-REP/MNT)	5.2, 11, 12, 13	CI, pinned configuration, independent reproduction.
Cost/legal/reporting (NFR-COST/LEG/RPT)	7.3, 10, 11, 12	Budget caps, licence notes, report with counts and limitations.

15. Risks, limitations, and open decisions

15.1 Risks and mitigations

Risk	Mitigation / evidence
Scope creep	Keep optional extensions deferred; prioritize authorization, evaluation, reliability, deployment.
Weak/bias labels	Document annotation guide, review samples, disclose benchmark size.
LLM judge bias/variance	Treat as secondary aid; report model/prompt and manual-review sample.
Access-control defect	Deny by default; filter + re-check; test direct, indirect, citation, cache and failure paths.
Prompt injection	Treat documents as untrusted; test categories; no tool actions from retrieved content.
API cost overrun	Budget caps, cached embeddings, bounded experiments, per-run estimates.
Licence violation	Check source terms; retain attribution; publish only permitted processed data.
Sensitive logs/secrets	Redact, minimize, scan secrets, set retention/access policy.
Unstable results	Freeze configuration and repeat selected runs.

15.2 Open decisions before implementation freeze

ID	Open decision	Proposed resolution / owner
OD-01	Final corpus and licence	Author to confirm source and redistribution terms before ingestion.
OD-02	Embedding, reranker, hosted LLM identifiers	Shortlist and benchmark in Week 1–2; record exact versions.
OD-03	Chunk size, overlap, top-N	Compare using fixed retrieval benchmark; freeze before final evaluation.
OD-04	Authentication/session token and expiry	Choose maintained password hashing + signed short-lived session token; document expiry and revocation.
OD-05	Latency/cost/quality targets	Set after baseline, before final run.
OD-06	Synthetic visibility assignment	Define reproducible assignment rules and distribution; document they are synthetic.
OD-07	Admin document-permission editing	Decide whether Admin can modify document permissions or only user permissions.
OD-08	Tracing tool and retention	Choose lightweight tracing; define access, redaction and retention.
OD-09	Escalation message and behavior	Define safe wording; no full case workflow in MVP.
OD-10	Cache	Prefer disabled initially; add only with permission-aware keys and invalidation tests.
OD-11	Request/API versioning and limits	Set API prefix, maximum question size, timeout, rate limit.
OD-12	Held-out evaluation split	Predefine split and avoid tuning on final set; state limitations if sample is too small.

15.3 Known limitations

The corpus uses public materials with synthetic visibility labels, not actual enterprise confidentiality labels. The demo does not integrate a real identity provider and is not multi-tenant hardened or certified. A small benchmark cannot support broad generalization claims. Hosted LLM variability, source licensing, model latency, and cost remain dependencies to measure. Prompt-injection resistance is evaluated only against documented attack scenarios.

Appendix A. Example query contract

A.1 Request validation

Field	Rule (proposed)
question	Required non-empty string; trim whitespace; enforce a configured maximum length.
request_options.include_citations	Boolean; defaults to true for the MVP.
session	Required authenticated session except on public login/health routes.
client role/permissions	Ignored for authorization; reject or ignore fields to prevent ambiguity.

A.2 Response schema

```
QueryResponse {
  request_id: string
  status: "ANSWERED" | "PARTIAL" | "ABSTAINED" | "ESCALATED" | "ERROR"
  answer: string
  citations: Citation[]
  verification: {
    supported_claims: integer
    removed_or_qualified_claims: integer
  } | null
}
Citation {
  chunk_id: string
  doc_id: string
  section: string
  source_url: string
  quote?: string  # only if authorized and required
}
```

A.3 Safe response principles

• Do not mention the existence, title, number, or content of inaccessible documents.

• Never return a citation that has not been resolved to an authorized chunk.

• Do not fabricate policy answers when retrieval or verification fails.

• Make abstention clear and actionable without implying that restricted evidence exists.

Appendix B. Design review checklist

• ■ All SRS functional requirements have a design location and test evidence.

• ■ Permission grants are loaded from server-side state and fail closed.

• ■ Retrieval filters are applied in Weaviate and candidates are rechecked before context assembly.

• ■ Every citation/source endpoint performs authorization independently.

• ■ Cache is disabled or permission-aware, version-aware and invalidated on permission changes.

• ■ Retrieved content is treated as untrusted; no tool actions are driven by it.

• ■ Claim verification and abstention are measured, including false refusals.

• ■ SEC-01 to SEC-10 are automated where practical and reported with counts.

• ■ Logs and traces exclude secrets and restricted passage text.

• ■ Corpus licence, attribution and redistribution terms are confirmed.

• ■ Model IDs, prompt versions, package versions, dataset split and run configuration are pinned.

• ■ Baseline and final metrics include quality, security, p50/p95 latency and estimated cost.

• ■ Hosted demo uses HTTPS, synthetic users and public content only.

• ■ Open decisions and limitations are visible in README and final report.

End of document • PolicyPilot SDD v1.0